

CARRERA

Sistemas y Gestión en Data

TEMA

**Adivina mi Dibujo: Sistema de Clasificación de Dibujos en Tiempo Real
Mediante una Red Neuronal Convolucional**

Evaluación 2

AUTOR

Lizbeth Alexandra Chacasaguay Yugcha

ASIGNATURA

Inteligencia Artificial

Adivina mi Dibujo: Sistema de Clasificación de Dibujos en Tiempo Real Mediante una Red Neuronal Convolutiva

Problema Abordado

El proyecto Adivina mi Dibujo consiste en desarrollar un sistema que, mientras el usuario dibuja un objeto sobre un lienzo con el mouse, predice en tiempo real a qué categoría pertenece el dibujo, entre un conjunto de cinco categorías: gato, casa, sol, bicicleta y paraguas. El sistema actualiza su predicción de forma continua mientras se dibuja, no únicamente al finalizar el trazo, y muestra las tres predicciones más probables junto con su porcentaje de confianza.

Esta modalidad de proyecto fue elegida considerando el hardware disponible (sin cámara web confiable) y la posibilidad de construir una interfaz simple basada en mouse, además de representar un problema con menor riesgo de ambigüedad frente a otras opciones del curso, como la clasificación de expresiones faciales.

Dataset Utilizado

Se utilizó el dataset público Quick, Draw! de Google (Google Creative Lab, 2017), que contiene millones de dibujos simplificados de 28×28 píxeles en escala de grises, aportados por usuarios de todo el mundo y organizados por categoría.

- Categorías seleccionadas: gato, casa, sol, bicicleta y paraguas, elegidas por presentar siluetas visualmente distintas entre sí, reduciendo el riesgo de confusión entre clases.
- Tamaño del dataset: 2,000 imágenes por categoría (10,000 imágenes en total), cumpliendo el mínimo solicitado en la tarea.

- División: 80% para entrenamiento (8,000 imágenes) y 20% para prueba (2,000 imágenes), mezcladas aleatoriamente antes de dividir (semilla fija = 42) para asegurar representación balanceada de las cinco clases en ambos conjuntos.
- Preprocesamiento: normalización de los valores de píxel del rango original (0–255) al rango (0–1), dividiendo entre 255.

Arquitectura del Modelo

Se entrenó una red neuronal convolucional (CNN) desde cero. No se utilizó Transfer Learning, dado que los dibujos de Quick, Draw! son trazos simplificados de 28×28 píxeles y no fotografías reales, por lo que una red preentrenada en ImageNet no ofrece ventaja para este tipo de entrada (Chollet, 2021).

Conv2D(32, 3×3 , ReLU) $\rightarrow$ MaxPooling2D(2×2) $\rightarrow$ Conv2D(64, 3×3 , ReLU) $\rightarrow$

MaxPooling2D(2×2) $\rightarrow$ Flatten $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(5, Softmax)

- Dos bloques de Conv2D y MaxPooling: el primero detecta patrones básicos, como bordes y curvas; el segundo combina esos patrones en formas más complejas propias de cada categoría.
- Una capa Dropout con probabilidad de 0.3, que apaga aleatoriamente el 30% de las neuronas durante el entrenamiento para reducir el riesgo de sobreajuste, dado el tamaño moderado del dataset.
- Una capa de salida con activación Softmax y cinco neuronas, una por categoría, correspondiente a un problema de clasificación multiclase.
- Entrenamiento de 15 épocas con el optimizador Adam y la función de pérdida `sparse_categorical_crossentropy`, reservando 10% de los datos de entrenamiento para validación.

Métricas de Desempeño

El modelo fue evaluado sobre el conjunto de prueba, compuesto por 2,000 imágenes que el modelo no utilizó durante el entrenamiento. La Tabla 1 resume los resultados obtenidos.

Tabla 1

Resultados de Evaluación en el Conjunto de Prueba

Métrica	Valor
Accuracy (prueba)	96.58%
Loss (prueba)	0.1549
Meta mínima del proyecto	75%

Tabla 2

Matriz de Confusión del Conjunto de Prueba

Real \ Predicción	Gato	Casa	Sol	Bicicleta	Paraguas
Gato	386	1	7	3	3
Casa	10	370	0	0	1
Sol	5	2	385	2	3
Bicicleta	8	1	1	409	0
Paraguas	2	1	8	2	388

Las confusiones más frecuentes ocurrieron entre sol y paraguas (8 casos) y entre gato y sol (7 casos). Esto se explica porque, en trazos rápidos y simplificados, ambas categorías comparten una forma curva o circular base con líneas saliendo (rayos, varillas u orejas), lo que dificulta su distinción cuando el trazo es muy esquemático.

Decisiones de Diseño

- Se limitó el dataset a 2,000 imágenes por clase, en lugar de utilizar los cientos de miles disponibles, para mantener el entrenamiento balanceado entre clases y con un tiempo de entrenamiento razonable en Google Colab.
- Se seleccionaron categorías con siluetas claramente distintas entre sí, minimizando de antemano el riesgo de un accuracy bajo por ambigüedad entre clases.

- La interfaz se construyó con Gradio (Abid et al., 2019) y su componente Sketchpad, utilizando el parámetro `live=True` para que la predicción se actualice automáticamente mientras se dibuja, sin necesidad de un botón de envío, cumpliendo el requisito de predicción en tiempo real.
- Fue necesario invertir los colores de la imagen capturada por el Sketchpad, ya que este dibuja con trazo oscuro sobre fondo claro, mientras que Quick, Draw! representa sus imágenes con el trazo en blanco sobre fondo negro. Sin este ajuste, el modelo recibiría la imagen de forma opuesta a como aprendió durante el entrenamiento.
- Se utilizó el componente `gr.Label(num_top_classes=3)` para mostrar las tres predicciones más probables junto con su porcentaje de confianza, cumpliendo el requisito de retroalimentación visual inmediata.

Referencias

- Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A., & Zou, J. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. arXiv. <https://arxiv.org/abs/1906.02569>
- Chollet, F. (2021). Deep learning with Python (2nd ed.). Manning Publications.
- Google Creative Lab. (2017). Quick, Draw! Dataset [Conjunto de datos]. GitHub. <https://github.com/googlecreativelab/quickdraw-dataset>
- TensorFlow Developers. (2024). TensorFlow (Versión 2.x) [Software]. Zenodo. <https://www.tensorflow.org>